



Projet n°8 : “Déployez un modèle dans le Cloud”

Soutenance de Projet
2025-07-02
Gaillot Dimitri

SOMMAIRE

01

CONTEXTE

Rappel de la problématique,
jeu de données

02

MODELE

Environnement,
modélisation, PySpark

03

CLOUD COMPUTING

Fonctionnement détaillé

04

ENV Big Data

Environnement AWS EC2,
configuration

05

PySpark

Exécution Cloud,
résultats

01

CONTEXTE



Start-up AgriTech “Fruits!”

Objectifs



Fruits!

- 1 Mettre en place un **modèle** de classification des images de fruits : tester le modèle fourni en local.
- 2 Passage à une architecture **Big Data** : mise en place de l'environnement Big Data **AWS EC2** (Amazon Web Services, Elastic Compute Cloud).

- **Nom** : Fruits-360
- **Origine** : Kaggle, Mihai Oltean (auteur)
- **90 483** images au **total**
- **67 928** images à **l'entraînement** (75% total)
- **22 688** images au **test** (25% total)
- **131** classes (fruits & légumes différents)
- **Taille** d'image : **100x100** pixels





02

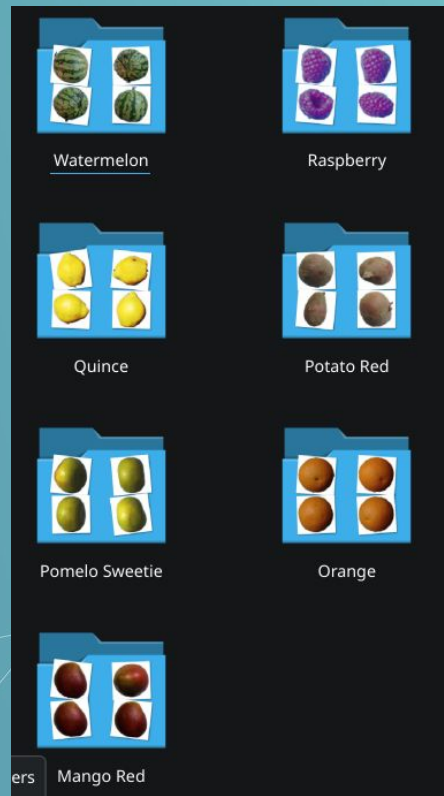
Modèle local

J'ai réduit le dataset initial à un dataset d'essai de **5 catégories** d'environ **150 images** chacune pour un total de **1094 images**, dans un soucis de rapidité d'exécution.

Environnement local :

- Linux (arch)
- **Java (jdk) 17** `openjdk version "17.0.15" 2025-04-15`
 - Identification version compatible, **Installation**
 - **Export .bashrc** pour rendre l'exécutable disponible aux processus (dont jupyterlab)
- Reprise du notebook fourni + ajout PCA

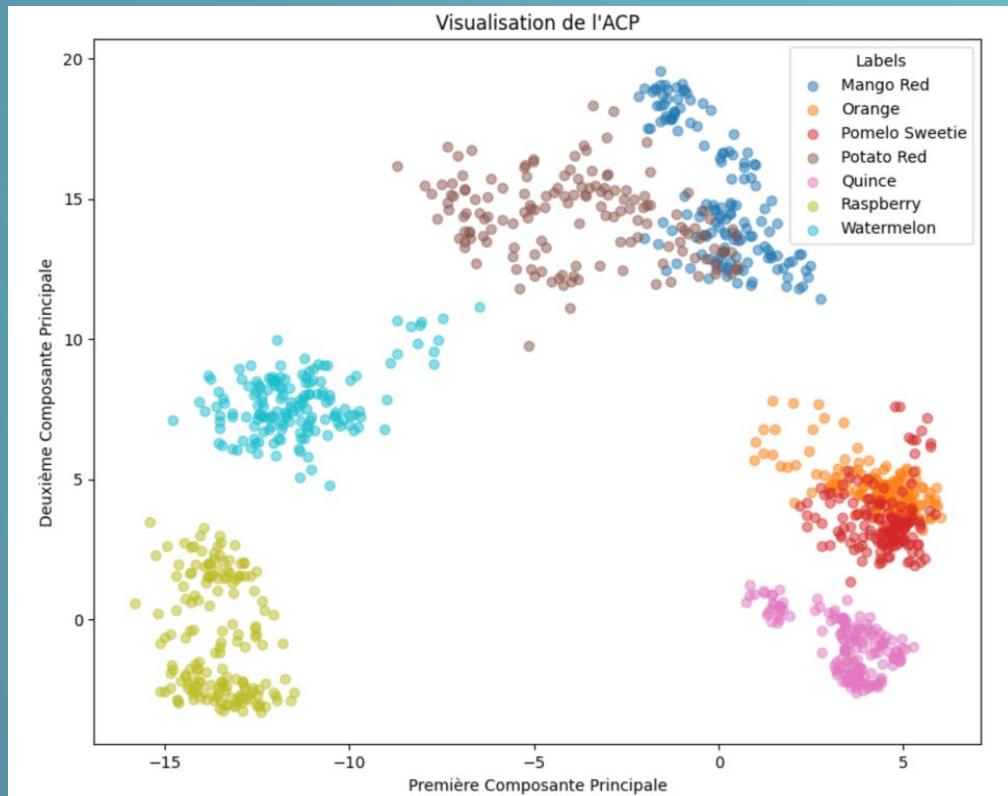
```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
```



Résultats de la modélisation

- **Agrégation** des résultats au format **Parquet**
- Calcul **PCA + Visualisation**

=> On retrouve sur les deux composantes principales des **clusters** relativement **séparés** des différents fruits & légumes de test.



The background is a solid teal-blue color. Overlaid on this are several abstract geometric patterns. These consist of white dots (nodes) connected by thin white lines (edges), forming a network or mesh structure. Some of these structures are more dense and complex, while others are simpler, consisting of a few connected dots and lines. There are also some isolated dots and small, faint triangular shapes scattered across the background.

03

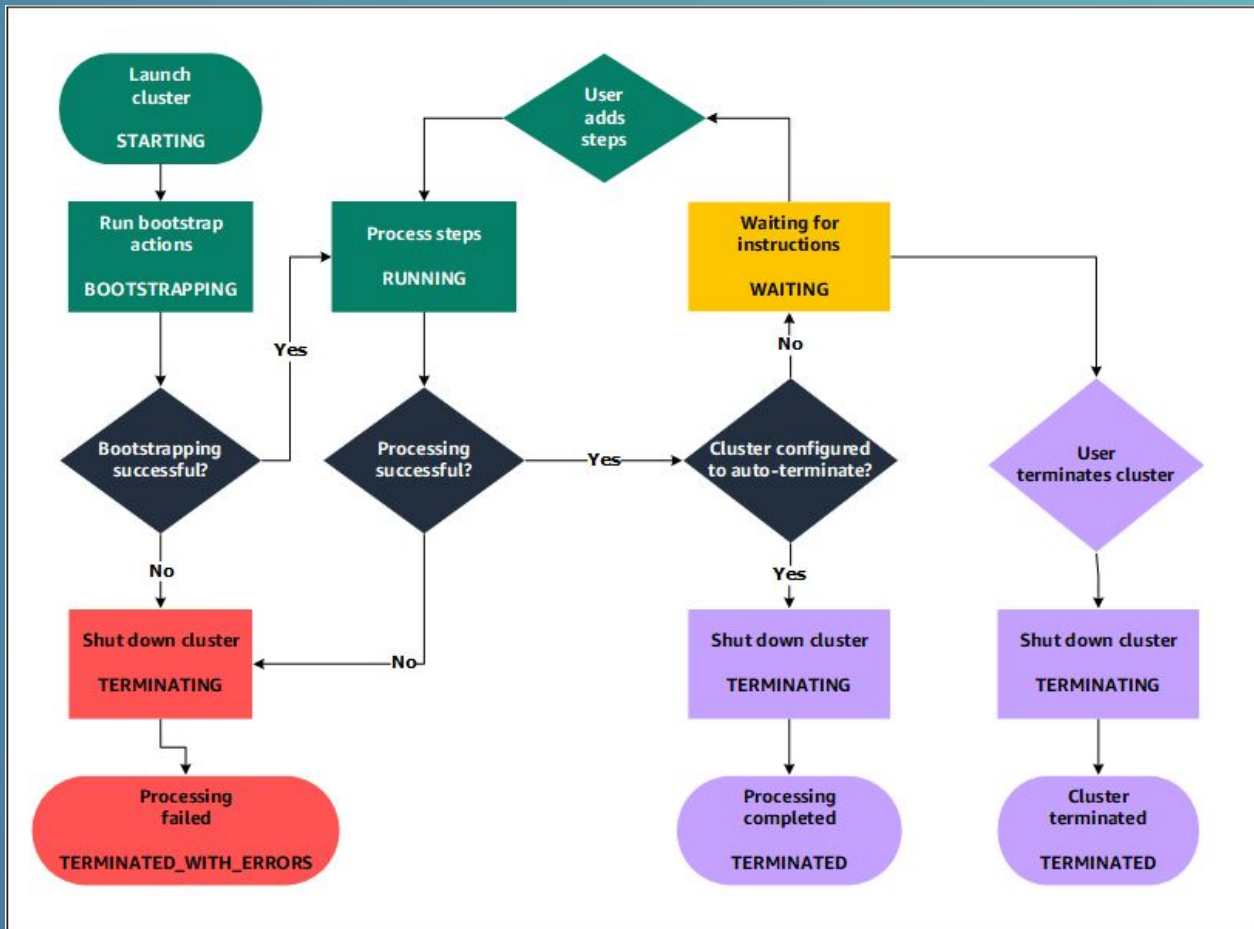
Cloud Computing

AWS : EMR sur EC2

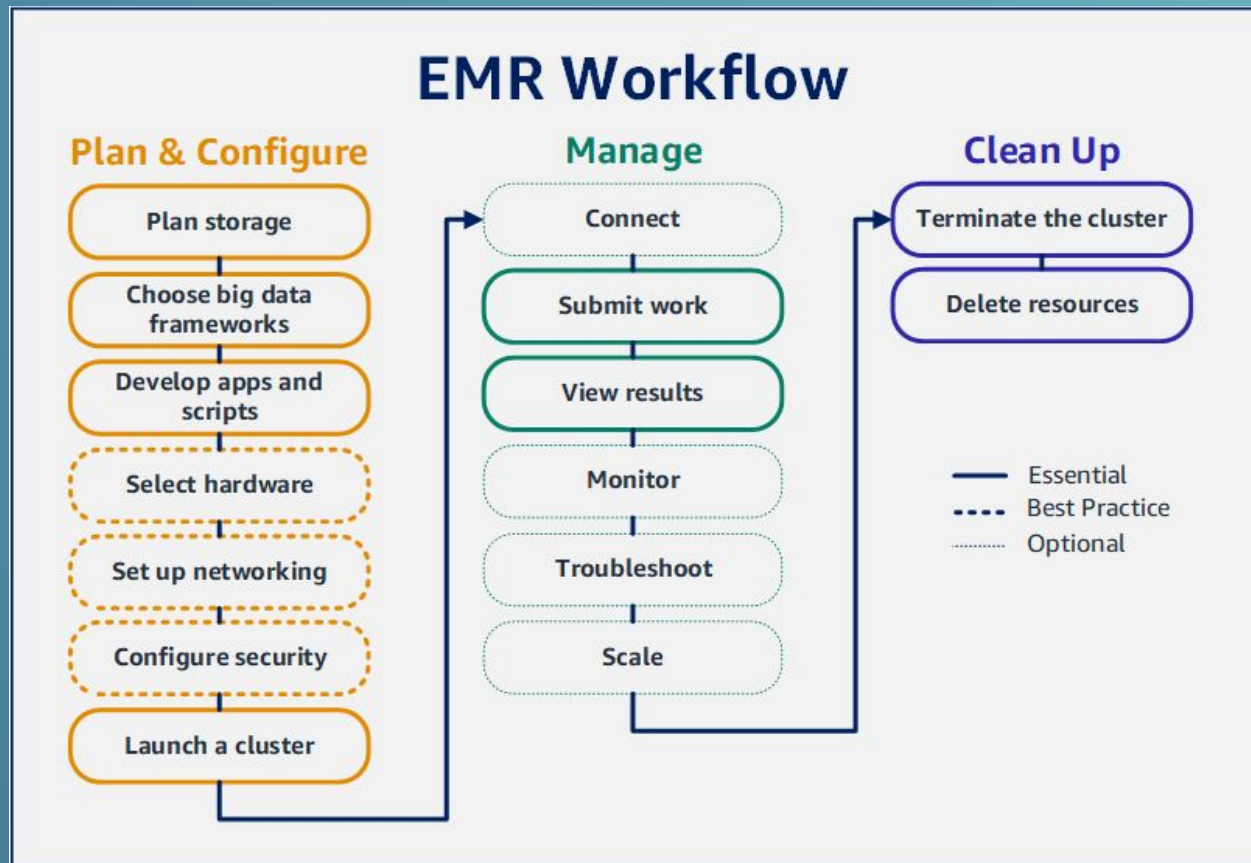
- AWS, **Amazon Web Services** : plateforme de service cloud Amazon (calcul, stockage, BDD...)
 - gère **l'approvisionnement**, la **configuration** et **l'optimisation** des ressources.
 - fournit l'intégration avec ses autres services comme **S3** (Simple Storage Service / **stockage**), Redshift (BDD, Data Warehouse)...
- EC2, **Elastic Cloud Computing** : service de location de serveurs virtuels (instances) avec capacité de calcul **redimensionnable** (CPU, mémoire, OS, logiciels...).
 - Evolutif : ajout ou retrait d'instances selon la charge de travail.
 - Coût : ajuster les ressources au plus près des besoins.
- EMR, **Elastic MapReduce**, service géré facilitant le traitement de données en grande quantité
 - via **frameworks** open source : Hadoop, **Spark**, Hive, Presto, Flink...
 - utilise des instances **EC2** comme noeuds de calcul.



EC2 : cycle de vie d'un cluster



Processus EC2 typique



Pyspark : Caractéristiques

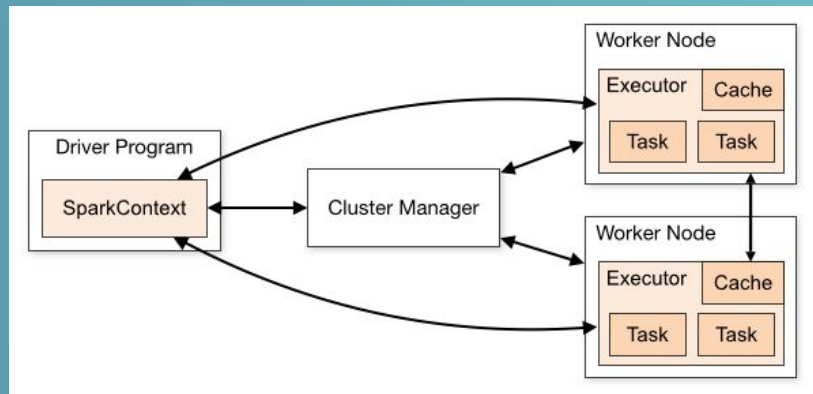


- **Evolutivité** : traitement de grands ensembles de données via **répartition** sur des clusters
- **Performance** : traitement à grande vitesse grâce au calcul en **mémoire cache**
- **Facilité d'usage** : API Python accessible et variée : **Java, Scala, Python, R, SQL**
- **Tolérance aux pannes** : récupération automatique des données, robustesse du **RDD (Resilient Distributed Dataset)**, la structure de données utilisée :
 - immuable** (non modifiable), **distribué** (partitionné & distribué sur les noeuds d'un cluster), **résilient** (lignage permet la reconstruction de données en cas de défaillance)
- **Plate-forme unifiée** : Inclut le **traitement** par paquets (batches), **l'analyse** de données, le **machine learning**
- **Traitement en temps réel** : gestion à la volée des flux de données
- **Machine learning** : algorithmes d'apprentissage adaptés via **ML lib**, permet l'entraînement et le déploiement à toute échelle
- **Support communautaire** : documentation étendue et assistance communautaire

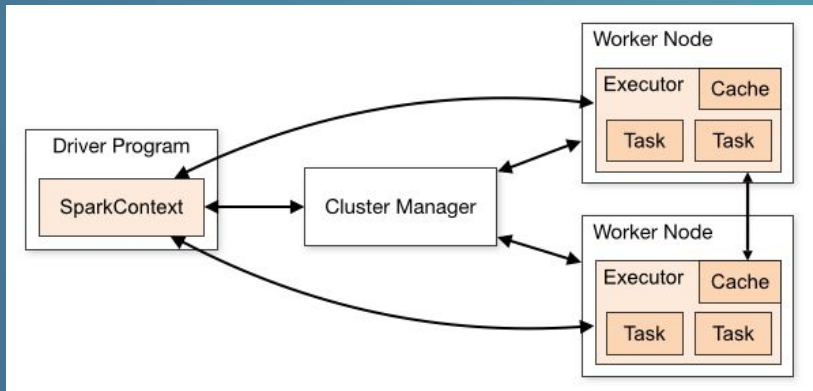
PySpark : architecture

1) Composants principaux :

- **Driver Program** : processus principal qui exécute la fonction *main()* de l'application et crée l'objet *SparkContext*. **Coordonne** l'exécution des tâches sur le cluster
- **Cluster Manager** : service externe pour l'**acquisition de ressources** sur le cluster. Spark peut utiliser différents types de gestionnaires (YARN, Kubernetes, gestionnaire de Spark)
- **Executors / Worker Nodes** : les processus lancés sur les noeuds du cluster qui **exécutent les calculs** et **stockent les données** pour l'application. Chaque noeud obtient ses propres processus d'exécution qui restent actifs durant toute la durée de vie de l'application.



PySpark : architecture



2) Fonctionnement :

- Le *SparkContext* **se connecte** à un cluster manager pour **obtenir des ressources**.
- *Spark* **envoie le code** de l'application (Python ou autre) aux **noeud de travail**.
- Le *SparkContext* envoie des tâches aux noeuds de travail pour exécuter les calculs

3) Points Clés

- Chaque application a ses **propres processus d'exécution** (applications isolées)
- Spark est **agnostique pour le Cluster Manager** (différentes solutions)
- Le programme **driver doit être accessible** en réseau **depuis les noeuds de travail**
- Le driver doit être exécuté près des noeuds workers, **idéalement sur le même réseau local**



04

Environnement AWS EC2

Configuration cluster

Pour plus de simplicité, nous allons nous focaliser sur le résumé de configuration du cluster EMR sur EC2 :

- **Nom** du cluster
- Version **EMR** (Elastic Map Reduce)
- Lot **d'applications** : **JupyterHub, Spark, TensorFlow**
- **Configuration** cluster :
 - **Uniforme** m5.xlarge pour les nodes primaire, coeur et de travail
 - **Quantité** d'instances : modifiable, minimale ici
- Clusters : **échelle** & **provisionnement** : minimum (1 instance)

Summary [Info](#)

Name and applications - *required*

Name

P8_spark_cluster

Amazon EMR release

emr-7.9.0

Application bundle

Custom (JupyterHub 1.5.0, Spark 3.5.5, TensorFlow 2.16.1)

Cluster configuration - *required*

Uniform instance groups

Primary (m5.xlarge), Core (m5.xlarge), Task (m5.xlarge)

Cluster scaling and provisioning - *required*

Provisioning configuration

Core size: 1 instance

Task size: 1 instance

Configuration (suite)

- **Node configuration** : configuration pointant vers le bucket S3 pour jupyter

▼ Node configuration - optional

☒ Enter configuration ☐ Load J

```
1 [
2 {
3   "classification": "jupyter-s3-conf",
4   "properties": {
5     "s3.persistance.bucket": "p8-data-dgdev",
6     "s3.persistance.enabled": "true"
7   }
8 }
9 ]
```

- **Bootstrap actions** : fichier .sh contenant les actions d'installation des librairies Python

▼ Bootstrap actions (1) Info

Use bootstrap actions to install software or customize your instance configuration.

Remove Edit Add

Name	Amazon S3 location	Arguments
☐ bootstrap env python librairies	s3://p8-data-dgdev/bootstrap-emr.sh	-

```
projet_8 > $ bootstrap-emr.sh
1 sudo python3 -m pip install -U setuptools
2 sudo python3 -m pip install -U pip
3 sudo python3 -m pip install wheel
4 sudo python3 -m pip install pillow
5 sudo python3 -m pip install pandas==1.2.5
6 sudo python3 -m pip install pyarrow==4.0.1
7 sudo python3 -m pip install boto3==1.26.0
8 sudo python3 -m pip install s3fs==2023.1.0
9 sudo python3 -m pip install fsspec==2023.1.0
10 sudo python3 -m pip install namex
11 sudo python3 -m pip install rich
12 sudo python3 -m pip install dm-tree
```

Configuration (suite)

- **Réseau** : configuration par défaut, permet de relier les noeuds entre-eux
 - **Pilotage** des noeuds de travail
 - Remontée des **résultats**
- **Primary node security** : nécessite l'ouverture de ports UDP pour l'**accès SSH** (JupyterHub)
- **Coupure** de la **grappe** (cluster) de serveurs : permet d'éviter de consommer des ressources "à vide"
- **Tags** : mots-clés descriptifs

Networking - required

VPC

vpc-062c00872... [↗](#)

Subnet

subnet-03feb9... [↗](#)

Primary node security group

sg-02c86f1243... [↗](#)

Core node security group

sg-0a13ab2590... [↗](#)

Cluster termination

Cluster termination

Terminate cluster after idle time

Idle time: 1 hour

Tags

Number of tags

1 tag

Configuration (suite)

- **Security configuration**, Amazon EC2 **key pair** :
clé de pairage permettant de se connecter sur l'instance via invite de commandes (cli).
- **IAM**, Identity & Access Management roles : rôles utilisateurs utilisés pour le service et la mise en place du cluster.
 - **Instance profile** (role) : nécessite l'ajout d'accès **S3** (Amazon Simple Storage Service, stockage cloud) pour **accéder aux données** d'entraînement & stocker les résultats.

Security configuration and EC2 key pair

Amazon EC2 key pair

key_pair_2025... [🔗](#)

Identity and Access Management (IAM) roles - *required*

Service role

AmazonEMR-ServiceRole-20250611T145754 [🔗](#)

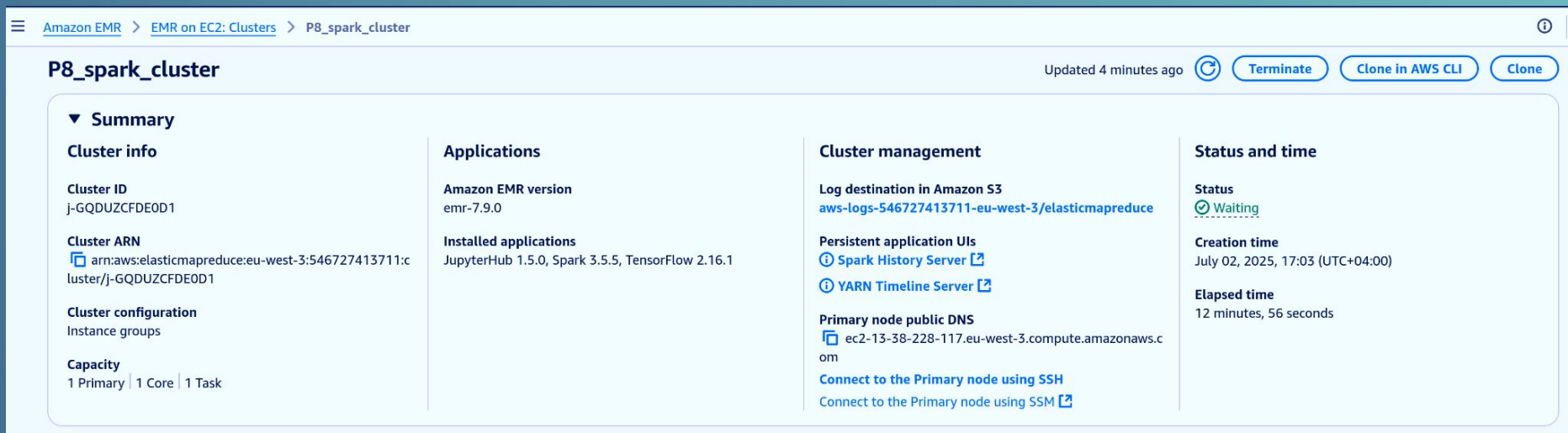
Instance profile

AmazonEMR-InstanceProfile-20250611T145736 [🔗](#)



Cluster fonctionnel :

- **Status** = Waiting
- **Primary node public DNS** : adresse publique pour connexion SSH



Amazon EMR > EMR on EC2: Clusters > P8_spark_cluster

P8_spark_cluster Updated 4 minutes ago [Refresh](#) [Terminate](#) [Clone in AWS CLI](#) [Clone](#)

▼ **Summary**

Cluster info Cluster ID j-GQDUZCFDE0D1 Cluster ARN arn:aws:elasticmapreduce:eu-west-3:546727413711:cluster/j-GQDUZCFDE0D1 Cluster configuration Instance groups Capacity 1 Primary 1 Core 1 Task	Applications Amazon EMR version emr-7.9.0 Installed applications JupyterHub 1.5.0, Spark 3.5.5, TensorFlow 2.16.1	Cluster management Log destination in Amazon S3 aws-logs-546727413711-eu-west-3/elasticmapreduce Persistent application UIs Spark History Server YARN Timeline Server Primary node public DNS ec2-13-38-228-117.eu-west-3.compute.amazonaws.com Connect to the Primary node using SSH Connect to the Primary node using SSM	Status and time Status Waiting Creation time July 02, 2025, 17:03 (UTC+04:00) Elapsed time 12 minutes, 56 seconds
---	--	--	--

05

PySpark

- **Upload** notebook + upload images

```
projet_8:zsh x projet_8:jupyter-lab x  
> aws s3 cp Prod_Pyspark.ipynb s3://p8-data-dgdev/jupyter/Prod_Pyspark.ipynb  
upload: ./Prod_Pyspark.ipynb to s3://p8-data-dgdev/jupyter/Prod_Pyspark.ipynb
```

- Connexion **SSH**

```
include .env  
  
aws_connect:  
@echo "Connecting to AWS instance..."  
ssh -i ${AWS_KEY} -D 5555 ${AWS_SSH_USER}@${AWS_SSH_HOST}
```

- **Proxy** local (FoxyProxy sur Firefox)

The screenshot shows the FoxyProxy web interface. At the top, there are buttons for 'Ajouter', 'filter', 'Ping', 'Tester', and 'Get Location'. Below these, there's a section for 'EMR' with a toggle switch and icons. The main configuration area has a table-like structure with fields for 'Nom ou Description (optionnel)', 'Type', 'Country', 'City', 'Couleur', 'Proxy DNS', 'Hostname', 'Port', 'Nom d'utilisateur (optionnel)', 'Mot de passe (optionnel)', and 'PAC URL'. The 'Type' is set to 'SOCKS5', 'Port' is '5555', and 'Nom d'utilisateur' is 'username'. There are also checkboxes for 'Store Locally' and a 'View' button. At the bottom, there's a 'Proxy by Patterns' section with a plus icon and a 'Save' button.

- Lien **JupyterHub** (login par défaut jovyan//jupyter)

- Démarrage de la session Spark

```
# L'exécution de cette cellule démarre l'application Spark
```

Starting Spark application

ID	YARN Application ID	Kind	State	Spark UI	Driver log	User	Current session?
0	application_1750851924477_0001	pyspark	idle	Link	Link	None	✓

SparkSession available as 'spark'.

- Définition des **dossiers S3**

```
PATH = 's3://p8-data-dgdev'
PATH_Data = PATH+'/Test'
PATH_Result = PATH+'/Results'
print('PATH: '+\
      PATH+'\nPATH_Data: '+\
      PATH_Data+'\nPATH_Result: '+PATH_Result)

FloatProgress(value=0.0, bar_style='info', desc=
PATH:      s3://p8-data-dgdev
PATH_Data:  s3://p8-data-dgdev/Test
PATH_Result: s3://p8-data-dgdev/Results
```

- Vérification des **données** (échantillon)

```
images.show(5)
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layo
+-----+-----+-----+-----+
|          path|  modificationTime|length|          content|
+-----+-----+-----+-----+
|s3://p8-data-dgde...|2025-06-11 08:04:50| 7353|[FF D8 FF E0 00 1...|
|s3://p8-data-dgde...|2025-06-11 08:04:50| 7350|[FF D8 FF E0 00 1...|
|s3://p8-data-dgde...|2025-06-11 08:04:50| 7349|[FF D8 FF E0 00 1...|
|s3://p8-data-dgde...|2025-06-11 08:04:50| 7348|[FF D8 FF E0 00 1...|
|s3://p8-data-dgde...|2025-06-11 08:04:52| 7328|[FF D8 FF E0 00 1...|
+-----+-----+-----+-----+
only showing top 5 rows
```

- Logs des jobs Spark

Spark Jobs (?)

User: livy
Total Uptime:
Scheduling Mode: FIFO
Completed Jobs: 4

▶ Event Timeline

▼ Completed Jobs (4)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id (Job Group) ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
3 (16)	Job group for statement 16 parquet at NativeMethodAccessorImpl.java:0	2025/06/25 12:01:39	1.0 min	1/1 (1 skipped)	24/24 (35 skipped)
2 (16)	Job group for statement 16 parquet at NativeMethodAccessorImpl.java:0	2025/06/25 12:01:01	38 s	1/1	35/35
1 (6)	Job group for statement 6 showString at NativeMethodAccessorImpl.java:0	2025/06/25 12:00:46	0.1 s	1/1	1/1
0 (5)	Job group for statement 5 showString at NativeMethodAccessorImpl.java:0	2025/06/25 12:00:37	2 s	1/1	1/1

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

- **S3** : Dossier **/Results** :
 - Indication **_SUCCESS**
 - Ajout de données au format **.parquet**

[Amazon S3](#) > [Buckets](#) > [p8-data-dgdev](#) > [Results/](#)

Results/

Objects

Properties

Objects (25)



Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions.

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	_SUCCESS	-	June 25, 2025, 16:02:41 (UTC+04:00)	0 B	Standard
☐	part-00000-1798942d-49cd-4c10-836a-891c5967422e-c000.snappy.parquet	parquet	June 25, 2025, 16:01:53 (UTC+04:00)	167.2 KB	Standard
☐	part-00001-1798942d-49cd-4c10-836a-891c5967422e-c000.snappy.parquet	parquet	June 25, 2025, 16:01:53 (UTC+04:00)	160.3 KB	Standard

Abstract geometric line art in the top corners of the slide, consisting of interconnected lines and dots forming various shapes.

Merci

Pour votre attention.

GAILLOT Dimitri